

Tutorial 6 - Token Reduction

Week 7 of INFS4205/7205 - Sem 1, 2026

Danny Wang

Recap from Lecture

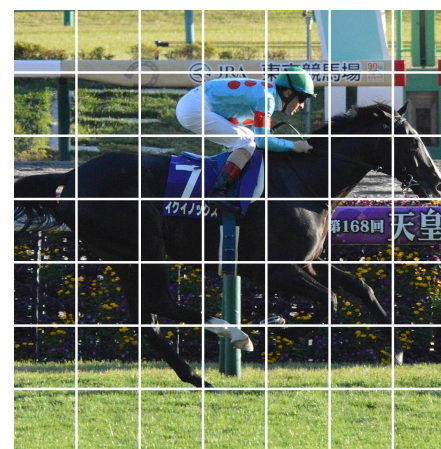
1. **Token Pruning:** select a subset of tokens
2. **Token Merging:** merge into smaller set of tokens
3. **Token Routing:** adaptive token selection (reuse previous unselected)
4. **Token Resampling:** learn a smaller set of latent tokens

Which one should we use?

How models convert images $\rightarrow$ tokens

How to calculate the number of tokens for a given image?

- Text $\rightarrow$ 1 word $\approx$ 1-2 tokens (autoregressive : auto + regressive)
- Image?



How models convert images → tokens

How to calculate the number of tokens for a given image?

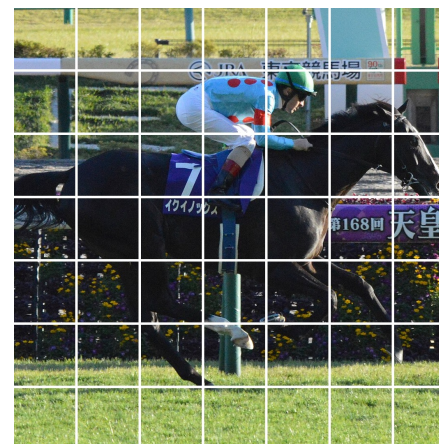
- Text → 1 word $\approx$ 1-2 tokens (autoregressive : auto + regressive)
- Image → tokens depend on resolution + model design

There is no single “token count” for an image, only a model-specific estimate.

How models convert images → tokens

Example of simple token counting :

- Image resolution: 1920 x 1080px
- Patch size: 16 × 16



$$\begin{aligned}\text{Token count} &\approx (\text{image width} / \text{patch size}) \times (\text{image height} / \text{patch size}) \\ &= (1920 / 16) \times (1080 / 16) \text{ (round up)} \\ &= 8160 \text{ (patches)}\end{aligned}$$

How models convert images → tokens

What if our model is restricted to a maximum of 6,144 patches?

- Effective token size: 32 x 32px per token
- 4 patches are pooled into a single token

How models convert images → tokens

What if our model is restricted to a maximum of 6,144 patches?

- Effective token size: 32 x 32px per token
- 4 patches are pooled into a single token

8160 > 6144 → **Resize**

- Resize to 1664 x 928 (maintains ~16:9 aspect ratio, divisible by 32)

How models convert images → tokens

What if our model is restricted to a maximum of 6,144 patches?

- Effective token size: 32 x 32px per token
- 4 patches are pooled into a single token

Num patches $\approx (1664 / 16) \times (928 / 16)$ (round up)
= 6032 (patches)

Num Tokens = $6032 / 4 = 1508$ tokens

How models convert images → tokens

Other factors can affect actual token counts in practice:

- Resolution Resizing
- Special Tokens: Some architectures add "newline" tokens at the end of each row of patches to help the model understand the 2D structure when it's flattened into a 1D sequence.
- E.g., OpenAI: compress patches or use adaptive tiling.

Higher resolution = more tokens

More detail = more tokens used internally

Why do we need token reduction?

What happens when image tokens increase in a VLM?

Why do we need token reduction?

What happens when image tokens increase in a VLM?

- Full self-attention cost scales as:
 - $\text{Cost} \propto (N_{\text{token}} + N_{\text{image}})^2$

Why do we need token reduction?

What happens when image tokens increase in a VLM?

- Full self-attention cost scales as:

- $\text{Cost} \propto (N_{\text{token}} + N_{\text{image}})^2$

Case	Total tokens	Attention cost
Text only (200)	200	40k
+ 200 image tokens	400	160k (4×)
+ 800 image tokens	1000	1M (25×)

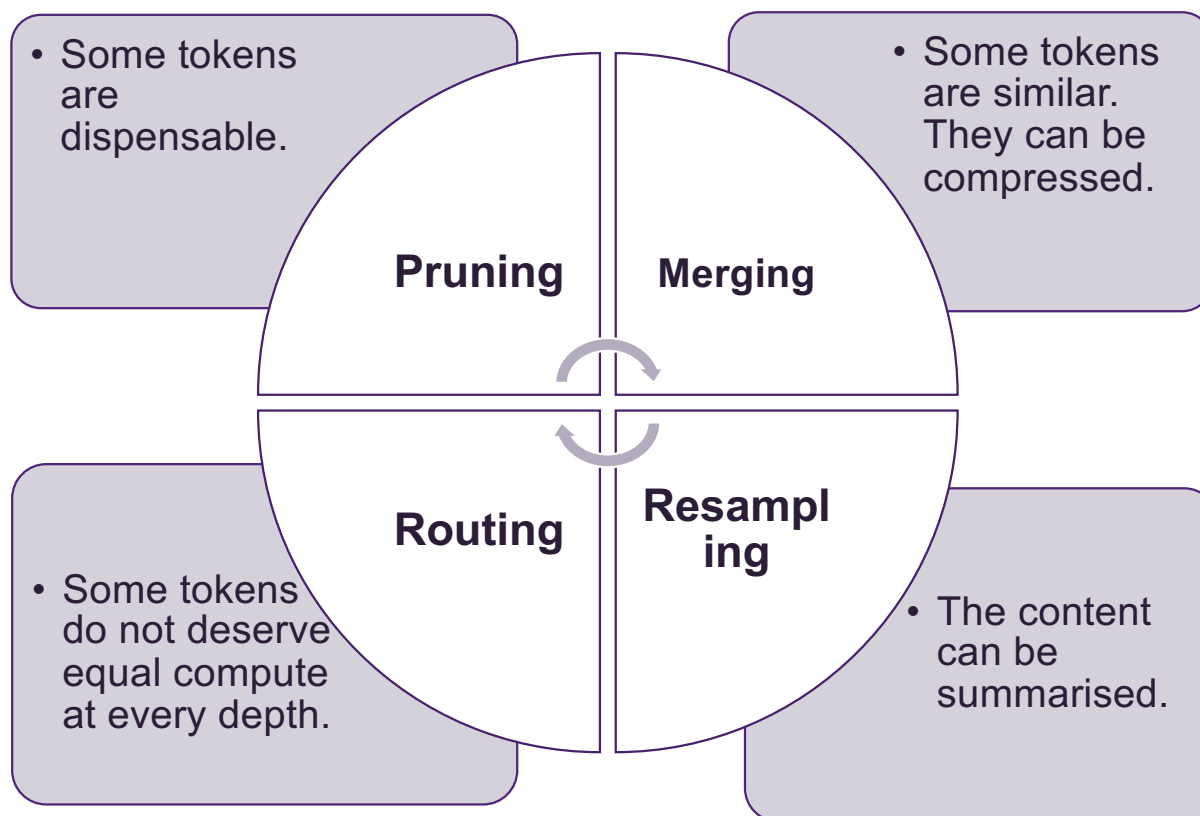
Why do we need token reduction?

What happens when image tokens increase in a VLM?

- Full self-attention cost scales as:
 - $\text{Cost} \propto (N_{\text{token}} + N_{\text{image}})^2$
- Doubling image tokens $\rightarrow \sim 4\times$ more compute
- High-resolution images quickly dominate cost

➤ Token reduction is necessary and essential

What kind of redundancy are you assuming?



Pruning

Q. Why does a good token-importance score still not guarantee safe pruning?

Pruning

Q. Why does a good token-importance score still not guarantee safe pruning?

A token can look **unimportant locally**, but still be **critical later** in the computation. Most importance scores are:

- attention-based → “who is used now”
- gradient-based → “who matters for loss now”



Pruning

Q. Why does a good token-importance score still not guarantee safe pruning?

A token can look **unimportant locally**, but still be **critical later** in the computation. Most importance scores are:

- attention-based → “who is used now”
- gradient-based → “who matters for loss now”

But for transformers (Multi-layer, non-linear):

- A token might not be used early
- becomes critical after interactions in deeper layers



Pruning

Q. Why does a good token-importance score still not guarantee safe pruning?

In VLMs: image patch tokens

- early layers → low attention
- later layers → fused into text → suddenly important

Pruning early can:

- remove “latent carriers of information”
- break future compositional reasoning

If a token has zero attention now, does that mean it has zero causal impact later? What scenarios does Token Pruning fail?

Merging

Q. Why is preserving more information not always the same as preserving the right information?

Merging

Q. Why is preserving more information not always the same as preserving the right information?

Merging keeps more tokens, but may mix signals in a way that hurts the task.

- Merges based on similarity / averages or aggregates features



Merging

Q. Why is preserving more information not always the same as preserving the right information?

Merging keeps more tokens, but may mix signals in a way that hurts the task.

- Merges based on similarity / averages or aggregates features

Even if total information is preserved:

- structure may be lost
- distinctions may blur

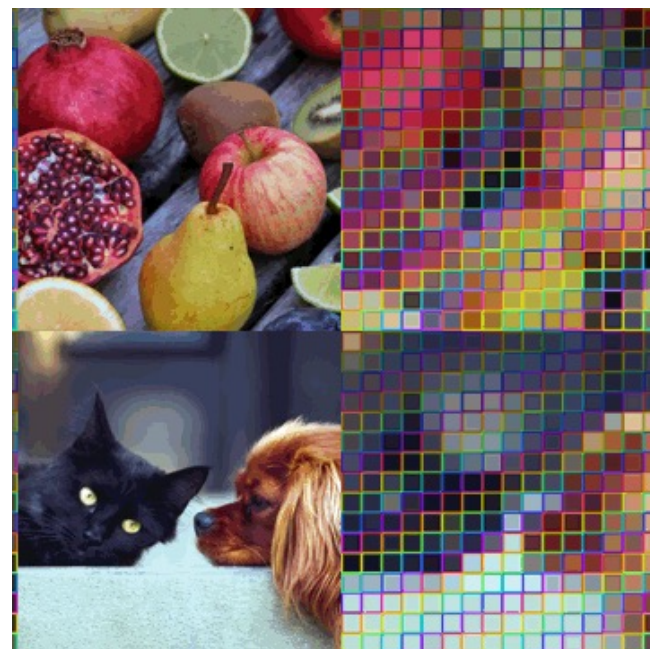


Merging

Q. Why is preserving more information not always the same as preserving the right information?

Example: two similar patches

- “cat ear” + “dog ear”
→ merged → ambiguous “animal ear”



Merging

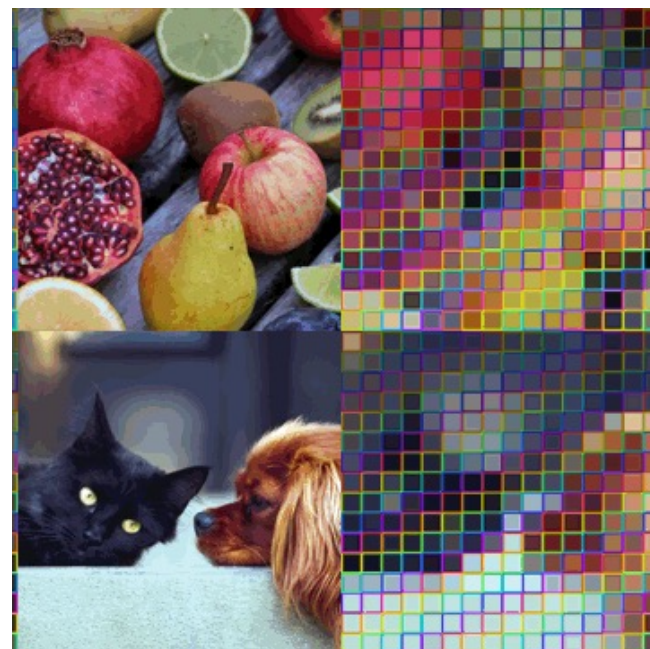
Q. Why is preserving more information not always the same as preserving the right information?

Example: two similar patches

- “cat ear” + “dog ear”
→ merged → ambiguous “animal ear”

So:

- entropy preserved $\neq$ task signal preserved



Merging

Q. Why is preserving more information not always the same as preserving the right information?

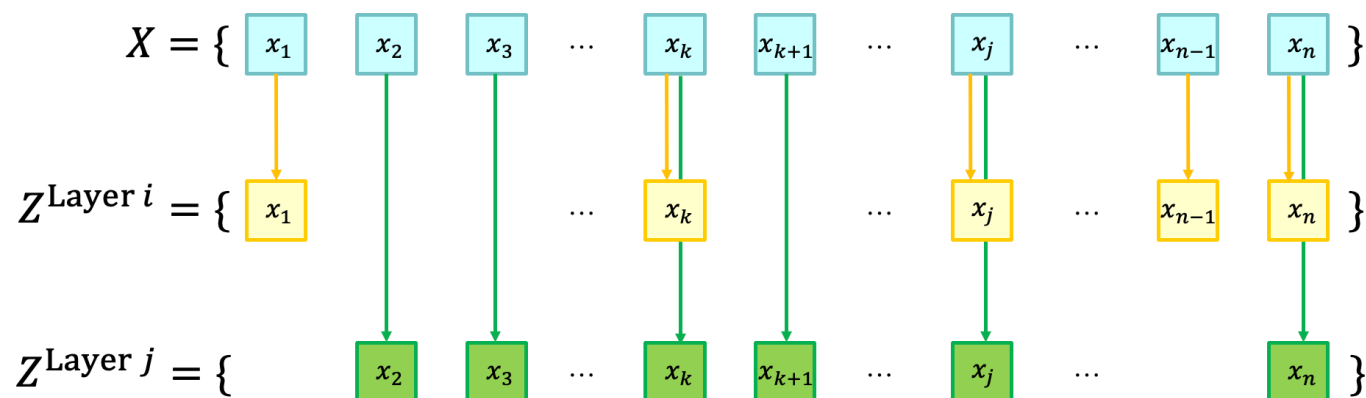
⚠ Practical implication

- Merging is safer than pruning for:
 - global understanding
- But worse for:
 - fine-grained classification

Routing

Q. Why is “not computing now” fundamentally different from “not needing the token”?

- Pruning → remove token → *information is gone*
- Routing → skip computation → *information is preserved*



Routing

What actually happens to information?

Pruning $\rightarrow$ remove token $\rightarrow$ *information is gone*

$Z = f(X_{keep}), X_{drop}$ is discarded

Directly reduces:

- input information $I(Z; X)$
- also reduces useful signal $I(Z; Y)$

Routing

What actually happens to information?

Routing $\rightarrow$ skip computation $\rightarrow$ *information is preserved*

$$Z_l = f_l(X_{active}), X_{inactive} \text{ is preserved}$$

- Only reduces compute, not information
- Tokens can be reused later

Token importance is not static:

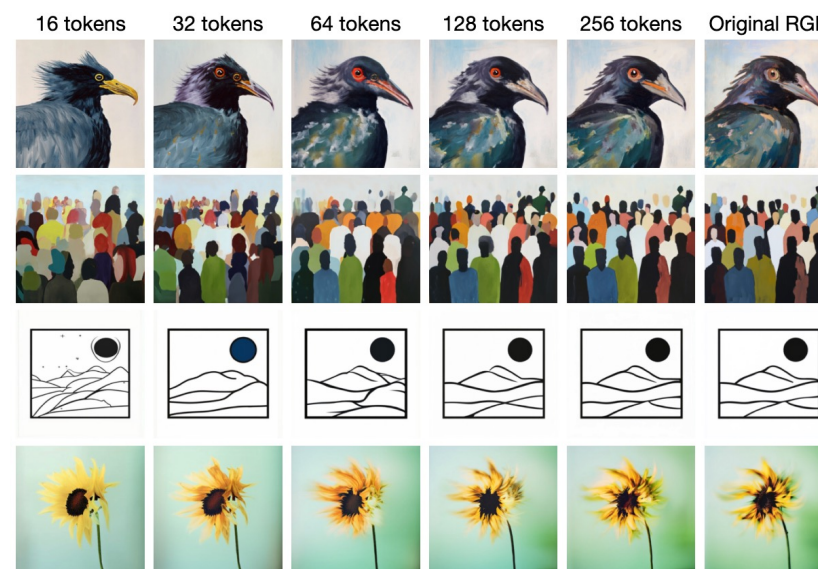
- Early layers $\rightarrow$ token looks useless
- Later layers $\rightarrow$ becomes critical

Resampling

Q. When is learning a latent query set better than deciding which original tokens to keep?

Sometimes it's better to summarize, not select.

- learn to extract task-relevant features



<https://flectok.epfl.ch/>

Resampling

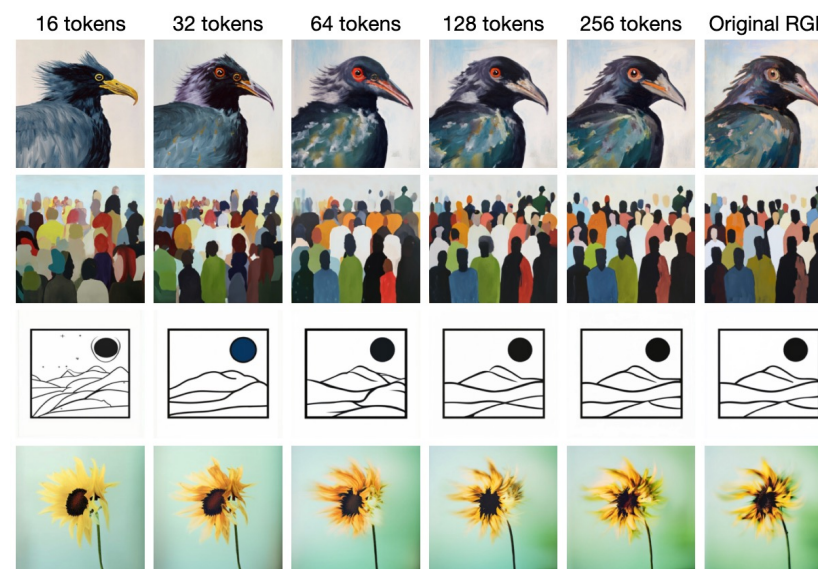
Q. When is learning a latent query set better than deciding which original tokens to keep?

Sometimes it's better to summarize, not select.

- learn to extract task-relevant features

Better when:

- structure is complex
- importance is distributed
- tokens interact globally
- strong generalisation



<https://flextok.epfl.ch/>

Information Bottleneck

Q. If two methods compress equally, why might one preserve task performance much better?

Information Bottleneck

Q. If two methods compress equally, why might one preserve task performance much better?

Compression ratio $\neq$ quality of retained information.

Both methods reduce 50% tokens

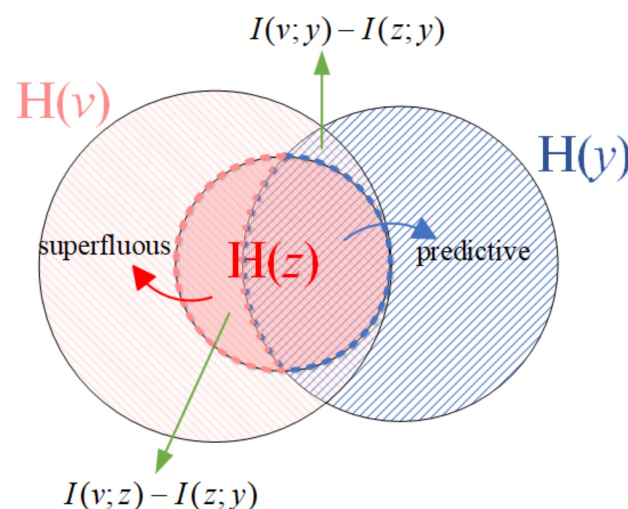
- Method A keeps random tokens
- Method B keeps task-relevant tokens

Information Bottleneck

Q. If two methods compress equally, why might one preserve task performance much better?

From IB view:

- goal is maximize:
 - $I(Z; Y)$ (task info)
- while minimizing:
 - $I(Z; V)$ (input info)



good method keeps predictive structure

bad method keeps irrelevant detail

Next Week

- **Multimodal RAG**

